UNIVERSIDAD DE COSTA RICA
Sede del Atlántico

RP Recinto de Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Quiz 6: Introducción al Desarrollo Móvil y Fundamentos de Flutter & Dart

Puntuación Total: 20 Puntos (5 Puntos por pregunta)
Tiempo Límite Recomendado: 25 minutos
Modalidad: Evaluación Individual Formativa / Sumativa
Instrucciones: Lea con atención cada uno de los cuatro enunciados de opción múltiple presentados a continuación. Seleccione únicamente la opción que considere correcta. Al concluir su resolución, consulte el solucionario analítico ubicado al final de este documento para verificar la validez técnica de su razonamiento.

Cuestionario de Evaluación

1. Arquitectura de Renderizado: Desarrollo Nativo vs. Enfoques Multiplataforma (5 Puntos)

En el análisis de viabilidad técnica para una aplicación empresarial con interfaces complejas que exige tasas de refresco fluidas (60 a 120 fotogramas por segundo), el equipo de ingeniería evalúa la diferencia arquitectónica fundamental entre **React Native** y **Flutter**. ¿Cuál de las siguientes afirmaciones describe con precisión técnica cómo gestiona Flutter el renderizado visual frente a React Native?

- [ ] A) Flutter utiliza un puente asíncrono (Bridge) en JavaScript que serializa mensajes JSON hacia los componentes nativos (OEM) del sistema operativo, mientras que React Native compila el código a binario puro en C++ omitiendo los componentes nativos.
- [ ] B) Flutter prescinde de los widgets y controles nativos del sistema operativo (OEM), renderizando cada píxel de la interfaz de forma autónoma mediante su propio motor gráfico (Impeller / Skia), a diferencia de React Native que mapea su lógica a vistas nativas de Android e iOS a través de un puente de comunicación.
- [ ] C) Flutter delega el dibujo de todos los componentes visuales en el motor WebKit del navegador interno del teléfono (WebView), mientras que React Native dibuja directamente en el buffer de gráficos sin intervención del sistema operativo.
- [ ] D) Ambos frameworks operan de forma idéntica convirtiendo el árbol de componentes a código fuente en Kotlin para Android y Swift para iOS en cada compilación en caliente (*Hot Reload*).

---

2. Ciclo de Vida del Sistema Operativo Móvil y Gestión de Memoria (5 Puntos)

A diferencia de una aplicación de escritorio o de servidor, una aplicación móvil opera bajo la supervisión estricta del sistema operativo anfitrión (Android o iOS), el cual puede pausar, detener o destruir procesos en cualquier momento. En el ciclo de vida de una pantalla en Android (*Activity*), ¿qué ocurre cuando el usuario recibe una llamada telefónica o abre otra aplicación que oculta completamente la pantalla actual, y qué buena práctica debe aplicar usted como desarrollador?

- [ ] A) La Activity pasa inmediatamente por `onDestroy()`, borrando toda la memoria RAM; el desarrollador debe reiniciar la aplicación desde cero cada vez que vuelva a primer plano.
- [ ] B) La Activity transiciona por los estados `onPause()` y `onStop()`; el desarrollador debe suspender animaciones pesadas, pausar suscripciones activas a sensores y persistir el estado temporal del usuario, evitando fugas de memoria o descargas innecesarias de batería.
- [ ] C) La aplicación entra en un bucle infinito en segundo plano consumiendo el 100% de la CPU hasta que el usuario decida cerrarla manualmente desde la barra de notificaciones.
- [ ] D) El sistema operativo transfiere la aplicación a la nube automáticamente mediante un hilo remoto, garantizando que el hardware no sufra interrupción alguna en su ejecución.

---

3. Rastreo de Código (Code Tracing) - Dart 3: Null Safety, Records y Pattern Matching (5 Puntos)

Analice detenidamente el siguiente fragmento de código escrito en **Dart 3**:

```
(String, int?) procesarUsuario(String? input, int edad) {
  final nombre = input ?? "Invitado";
  final puntos = edad >= 18 ? edad * 10 : null;
  return (nombre, puntos);
```

```
}

void main() {
  final (user, score) = procesarUsuario(null, 20);

  final resultado = switch (score) {
    null => 'Sin puntaje asignado',
    > 250 => 'Nivel Avanzado',
    >= 150 && <= 250 =>
      'Nivel Intermedio: $user ($score pts)',
    _ => 'Nivel Inicial: $user'
  };

  print(resultado);
}
```

¿Cuál es la salida exacta que se imprimirá en la consola al ejecutarse este programa?

- [ ] A) `Nivel Intermedio: Invitado (200 pts)`
- [ ] B) `Nivel Inicial: Invitado`
- [ ] C) `Sin puntaje asignado`
- [ ] D) Se produce un error de compilación debido a que la función `procesarUsuario` intenta retornar una tupla sin definir una clase contenedora formal.

---

4. Rendimiento y Concurrencia en Flutter: Event Loop frente a Isolates (5 Puntos)

Dart ejecuta el código de una aplicación en un único hilo principal (*UI Thread*) regulado por un bucle de eventos (*Event Loop*). Cuando una aplicación móvil debe procesar un archivo JSON masivo de 50 MB proveniente de una API o realizar un cálculo criptográfico intensivo, ¿cuál es el problema de rendimiento que se produce si se ejecuta dentro del hilo principal y cuál es la solución arquitectónica provista por Flutter?

- [ ] A) Ocurre un bloqueo del Event Loop que impide al motor gráfico dibujar los 60/120 fotogramas por segundo, provocando congelamiento visual (*jank*); la solución es delegar el procesamiento en un **Isolate** independiente (mediante la función `compute`), el cual corre en un hilo físico separado con su propia memoria aislada.
- [ ] B) Ocurre un desbordamiento de pila (*StackOverflowError*) irrecuperable en el navegador; la solución consiste en reemplazar Dart por JavaScript puro utilizando un Service Worker.
- [ ] C) El sistema operativo suspende la antena de red del dispositivo; la solución es invocar `setState()` repetidamente dentro de un ciclo `while(true)`.
- [ ] D) La palabra clave `await` resuelve automáticamente el problema creando hilos físicos nativos del procesador para cada función asíncrona, eliminando la necesidad de hilos secundarios.

Solucionario Analítico y Justificación Pedagógica

Pregunta 1: Arquitectura de Renderizado: Desarrollo Nativo vs. Enfoques Multiplataforma

- **Opción Correcta: B**
- **Justificación de la Opción Correcta:**
  Flutter revoluciona el desarrollo multiplataforma al no depender de los controles OEM (Original Equipment Manufacturer) del sistema operativo anfitrión. Incorpora su propio motor de renderizado gráfico de bajo nivel en C++ (originalmente Skia, y modernamente Impeller), lo que le permite dibujar cada widget píxel por píxel directamente sobre el lienzo del dispositivo a 60 o 120 FPS sin requerir un puente de serialización (*bridge*). Por el contrario, React Native mantiene una arquitectura donde el hilo de JavaScript se comunica a través de un puente con los componentes visuales nativos de Android (`android.widget.*`) e iOS (`UIKit`), lo que introduce sobrecarga de serialización en animaciones complejas.
- **Análisis de Opciones Incorrectas (Distractores):**
- **A es incorrecta:** Invierte totalmente los roles; es React Native el que utiliza un puente para comunicarse con componentes OEM, no Flutter.
- **C es incorrecta:** Describe la arquitectura de frameworks híbridos basados en WebView (como Apache Cordova o Ionic clásico), no la de Flutter.
- **D es incorrecta:** *Hot Reload* en Flutter inyecta código fuente actualizado directamente en la máquina virtual de Dart en tiempo de ejecución; no realiza una traducción estática a Kotlin ni a Swift.

Pregunta 2: Ciclo de Vida del Sistema Operativo Móvil y Gestión de Memoria

- **Opción Correcta: B**
- **Justificación de la Opción Correcta:**
  En la arquitectura móvil de Android, cuando una pantalla deja de ser visible porque otra aplicación o diálogo del sistema (como una llamada entrante) toma el primer plano, el sistema operativo invoca `onPause()` y posteriormente `onStop()`. En estos estados, la aplicación permanece en memoria pero debe suspender procesos demandantes (sensores GPS, animaciones, sockets o streams continuos) y almacenar el estado de la UI (mediante `Bundle` o almacenamiento local) para evitar pérdida de datos si el sistema decide terminar el proceso ante escasez de memoria RAM.
- **Análisis de Opciones Incorrectas (Distractores):**
- **A es incorrecta:** `onDestroy()` únicamente se ejecuta cuando la Activity finaliza explícitamente (ej. el usuario pulsa atrás) o cuando el sistema operativo necesita reclamar memoria de forma forzada; no ocurre de forma instantánea al recibir una llamada.

- **C es incorrecta:** Los sistemas operativos modernos cuentan con servicios de vigilancia (*Watchdog*) que suspenden o destruyen procesos que consumen recursos excesivos en segundo plano (ANR: *Application Not Responding*).
- **D es incorrecta:** La gestión de estados en segundo plano es una responsabilidad local del dispositivo móvil; el sistema operativo no traslada el hilo de ejecución a servidores remotos en la nube de forma automática.

Pregunta 3: Rastreo de Código - Dart 3

- **Opción Correcta: A**
- **Justificación de la Opción Correcta:**
  El análisis paso a paso de la ejecución revela:
- La función `procesarUsuario(null, 20)` recibe `input = null` y `edad = 20`.
- Mediante el operador de coalescencia nula `??`, `input ?? "Invitado"` evalúa a `"Invitado"`.
- En la expresión condicional ternaria, `edad >= 18` (`20 >= 18`) es verdadero, por lo que `puntos` toma el valor de `20 * 10 = 200`.
- La función retorna el **Record** `("Invitado", 200)`.
- En `main()`, la desestructuración por patrón `final (user, score) = ...` asigna `user = "Invitado"` y `score = 200`.
- La expresión `switch (score)` evalúa el valor `200`:
  - `null`: No coincide.
  - `> 250`: Falso ($200 \ngtr 250$).
  - `>= 150 && <= 250`: Verdadero ($150 \le 200 \le 250$).
- Retorna la interpolación `'Nivel Intermedio: Invitado (200 pts)'`.
- La salida impresa en consola es exactamente: `Nivel Intermedio: Invitado (200 pts)`.
- **Análisis de Opciones Incorrectas (Distractores):**
- **B es incorrecta:** El patrón comodín `_` actúa como rama por defecto (*default*), pero la condición anterior `>= 150 && <= 250` capturó con éxito el valor 200.
- **C es incorrecta:** `score` no es nulo; la edad 20 activó el cálculo numérico positivo.
- **D es incorrecta:** A partir de Dart 3.0, los **Records** son ciudadanos de primer orden en el sistema de tipos y no requieren definir clases auxiliares.

Pregunta 4: Rendimiento y Concurrencia en Flutter: Event Loop frente a Isolates

- **Opción Correcta: A**

- **Justificación de la Opción Correcta:**
  Dart opera bajo un modelo de concurrencia basado en un solo hilo (*Single-Threaded Event Loop*). Cuando se ejecutan instrucciones intensivas en CPU (como serialización masiva o procesamiento gráfico complejo), el bucle de eventos no puede procesar los eventos de renderizado visual a tiempo, ocasionando pérdidas de fotogramas (*dropped frames* o *jank*). Para resolver esto sin compartir memoria mutable (evitando condiciones de carrera y bloqueos mutuos), Dart y Flutter proveen **Isolates**, los cuales ejecutan tareas en núcleos físicos independientes con su propio montón de memoria (*heap*), comunicándose exclusivamente por paso de mensajes. La utilidad `compute()` de Flutter abstrae la creación y destrucción de un Isolate para tareas puntuales.
- **Análisis de Opciones Incorrectas (Distractores):**
- **B es incorrecta:** El código de Flutter en producción no corre sobre JavaScript ni en Service Workers (salvo en la compilación específica para Web); en dispositivos móviles se compila a código de máquina nativo AOT (ARM/x86).
- **C es incorrecta:** `setState()` dentro de un bucle `while(true)` saturaría de forma crítica el hilo de la UI, provocando que la aplicación deje de responder inmediatamente (*ANR*).
- **D es incorrecta:** `await` en Dart no crea hilos secundarios de CPU; simplemente pausa la ejecución de la función actual devolviendo el control al Event Loop hasta que el `Future` complete su resultado. Si el cálculo dentro de la función es costoso en CPU, congelará la interfaz independientemente del uso de `await`.